

RenewalVault

An Encrypted, Tamper-Evident Deadline & Renewal Tracking Platform

Project Title	RenewalVault
Submitted By	Mushkan
Course	Programming in Java
Submission Type	VITyarthi - Build Your Own Project
Degree Program	B.Tech CSE (Cyber Security & Digital Forensics)
University	VIT Bhopal University
Technology Stack	Java 21 (javax.crypto, java.util.zip, java.time, java.io)
Submission Date	12 September 2026

1. Introduction

RenewalVault is a command-line Java application built for the Programming in Java course to solve a problem that spans nearly every category of modern life: things with deadlines that are easy to forget. Subscriptions silently auto-renew and charge; insurance policies lapse unnoticed; a vehicle's compliance certificate expires; a tax filing deadline slips past; a warranty window closes right before an appliance breaks. Each of these is normally tracked -- if at all -- by a different single-purpose app, a sticky note, or human memory alone.

Rather than build another narrow reminder app, RenewalVault treats "don't forget this by this date" as one unified engineering problem across eleven life-domain categories, and uses it as a vehicle to apply a broad slice of core Java concepts in a single, coherent codebase: object-oriented design across five packages, generics (a type-safe `FileStore<T extends Serializable>` persistence layer), enums with behaviour (`Category.riskWeight()`), the Collections Framework (`PriorityQueue`, `List`, `Map`), the Streams API for search/filter/sort/analytics, the `java.time` date API for recurrence logic, object serialization and file I/O, and structured exception handling throughout. On top of that Java foundation, the project also builds a genuinely encrypted, password-protected vault (AES-256-GCM via `javax.crypto`, itself part of the standard JDK) and a SHA-256 hash-chained audit trail -- an ambitious, real-world feature choice that also happens to align with the author's Cyber Security & Digital Forensics specialization, rather than a requirement of this course.

The project is implemented entirely in Java 21 using only classes shipped with the standard JDK (`javax.crypto`, `java.util.zip`, `java.time`, `java.io`) -- no external libraries, no build tool, no internet connection required. It compiles and runs with nothing but `javac` and `java`.

2. Problem Statement

Everyday life is full of deadlines that are easy to forget: a subscription silently auto-renews and charges the user, an insurance policy lapses without notice, a warranty window closes right before an appliance breaks, a vehicle's insurance/PUC certificate expires, a routine health checkup gets pushed back for years, a tax or compliance filing is missed, groceries spoil unnoticed, and a plant dies because nobody remembered to water it on schedule.

Each of these is currently handled -- if at all -- by a *different* app, a sticky note, or simply human memory. There is no single, free, offline tool that treats this as one unified problem across every domain of a person's life, and none of the existing single-purpose tools give any real guarantee that the data they hold (insurance numbers, tax deadlines, personal notes) is protected at rest or safe from silent tampering.

2.1 Scope of the Project

RenewalVault is scoped as a fully offline, dependency-free, single-user terminal tool. It deliberately excludes multi-user accounts, cloud sync, push/SMS/email notifications, and third-party calendar/banking API integration, in favour of a self-contained core engine that solves the underlying tracking, prioritization, and data-protection problem well. Local AES-256 encryption and `.ics` calendar file export are included because both stay entirely offline and require no external service.

2.2 Target Users

- Individuals managing personal subscriptions, insurance, and warranties
- Vehicle owners tracking servicing and compliance deadlines
- Health-conscious individuals who want a reminder system for routine checkups

- Households wanting to reduce food waste by tracking grocery expiry dates
- Freelancers and small-business owners tracking tax/compliance filing deadlines
- Pet owners and plant enthusiasts tracking recurring care tasks
- Anyone who wants their personal deadline data actually protected, not stored in a plain text file

2.3 Objectives

- 1 Identify a meaningful, real-world problem (fragmented, unprotected deadline tracking) and design a single unified solution for it.
- 2 Apply core Java and object-oriented design concepts from the syllabus -- encapsulation, interfaces/enums with behaviour, generics, collections, exception handling, and file I/O -- to build a complete, working application rather than isolated exercises.
- 3 Implement the solution using Java's standard library end to end: the Collections Framework and Streams API for data structures and algorithms (priority queues, sorting, filtering), the java.time API for recurrence/date arithmetic, java.io/serialization for file-based persistence, and javax.crypto for applied cryptography.
- 4 Demonstrate understanding through structured documentation (this report, README, architecture and UML diagrams) and a rigorous, automated evaluation (a 12-case regression suite).

3. Functional Requirements

The brief requires at least three major functional modules with a clear input/output structure and a logical user workflow. RenewalVault implements **seven**:

#	Module	Responsibility	Key Inputs -> Outputs
1	VaultService	Item CRUD, search/filter/sort, undo	Item fields -> stored TrackedItem / query results
2	ReminderEngine	Risk-weighted urgency queue, hash-chained audit log, renewal streaks	Item list + logs -> ranked alerts, integrity verdict, streak counts
3	AnalyticsEngine	Budget analytics and ASCII charts	Active items -> spend charts, cost projections
4	ExportEngine	CSV backup/restore, .ics calendar export	Item list -> CSV/.ics file, or CSV file -> items
5	BackupEngine	Single-file encrypted vault backup/restore	./data folder -> .rvbackup file, and reverse
6	Security (CryptoUtil + VaultAuth)	AES-256-GCM encryption, PBKDF2 key derivation, password verification/rotation	Password + salt -> AES key; plaintext -> ciphertext
7	CLI & Reporting (Main)	Login flow, 18-option menu, input validation, colorized reporting	User keystrokes -> validated calls into every other module

Each module has a clear, testable input/output boundary (enforced by the automated test suite -- see Section 11), and the CLI layer provides a single logical workflow: *unlock vault* → *view dashboard* → *act on an item or the vault itself* → *persist* → *repeat or exit* (see the Workflow Diagram in Section 7).

Representative functional capabilities include:

- **CRUD operations** on tracked items across 11 categories (add, edit, deactivate, delete, undo)
- **Data input & processing:** recurrence math (one-time/weekly/monthly/yearly/custom-interval), risk scoring
- **Reporting & analytics:** urgency dashboard, category breakdowns, spend charts, cost projections, renewal streaks
- **Search, filter and multi-mode sort** across name, tag, category, overdue status, and due-date window

4. Non-Functional Requirements

Requirement	How RenewalVault addresses it
Security	AES-256-GCM encryption at rest (authenticated, tamper-evident by construction), PBKDF2-HMAC-SHA256 key derivation (120,000 iterations), constant-time password verification, no password or key ever persisted.
Reliability	A wrong password or corrupted file fails loudly and safely (GCM auth-tag rejection) instead of silently loading garbage data; every mutation is followed by an immediate encrypted persist.
Auditability / Logging	Every add, edit, renewal, snooze, and alert is recorded as a SHA-256 hash-chained log entry; verifyAuditIntegrity() can prove the entire history is untampered, or pinpoint the exact broken entry.
Usability	A single, consistent 18-option menu; colour-coded urgency (red/yellow/green/cyan); blank-to-keep-current-value editing; duplicate-name warnings; plain-language prompts throughout.
Maintainability	Clean package separation (model / engine / security / db / cli / test) with single-responsibility classes; every public method is documented; a 12-case regression suite guards against silent regressions during future changes.
Scalability (data volume)	Search, filter, sort, and the risk-ranked priority queue all operate in $O(n \log n)$ or better over the active item list, which comfortably scales to the thousands of items a single household or small business would realistically track.
Portability	Zero external dependencies beyond the JDK; a single encrypted .rvbackup file can move an entire vault to a different machine and be restored with nothing but the original password.
Error Handling Strategy	All user input parsing (dates, numbers, menu choices) is wrapped in try/catch with specific, actionable messages; cryptographic and I/O failures are caught at the FileStore boundary and never crash the running session.
Resource Efficiency	No background threads, no polling, no in-memory duplication of the item list beyond what is actively displayed; the entire vault for a realistic household fits in a few kilobytes of encrypted storage.

5. System Architecture

RenewalVault follows a layered architecture with a strict one-directional dependency flow: the CLI layer depends on the engine layer, the engine layer depends on the security and domain-model layers, and only the persistence layer touches disk directly -- and even then, only through the security layer's encryption. No layer reaches "upward" past the one above it.

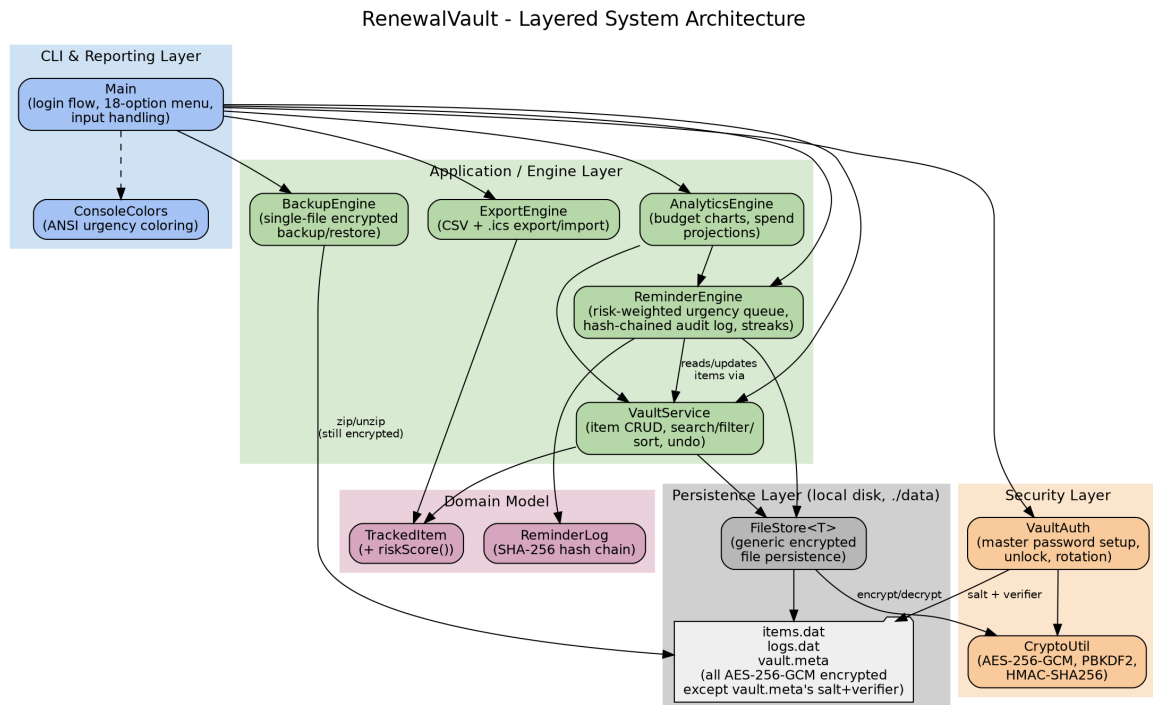


Figure 1: Layered system architecture, from the CLI down to encrypted disk storage.

This separation means, for example, that **FileStore** has no idea it is storing **TrackedItem** or **ReminderLog** objects specifically -- it is a generic **FileStore<T>** extends **Serializable** that encrypts and decrypts whatever it is given. This is what let a second data type (**ReminderLog**) and a completely new module (**BackupEngine**) be added later without touching the encryption code at all.

6. Design Diagrams

6.1 Process Flow / Workflow Diagram

RenewalVault - Process Flow / Workflow Diagram

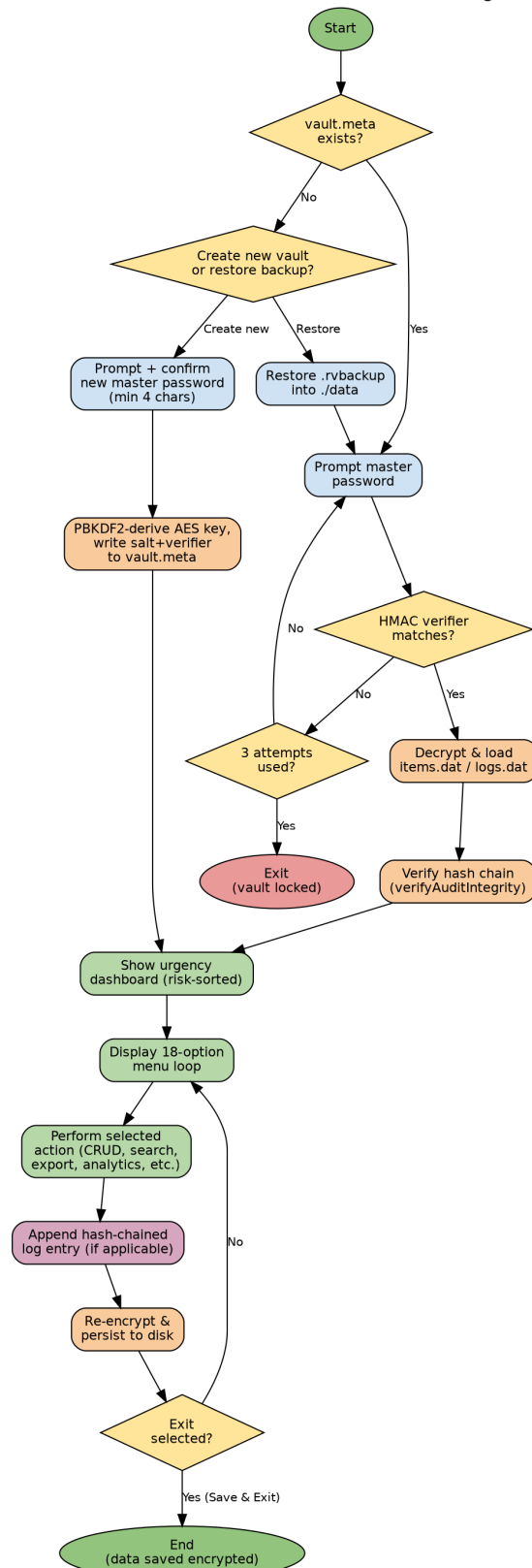


Figure 2: End-to-end process flow, from launch through the menu loop to save & exit.

6.2 Use Case Diagram

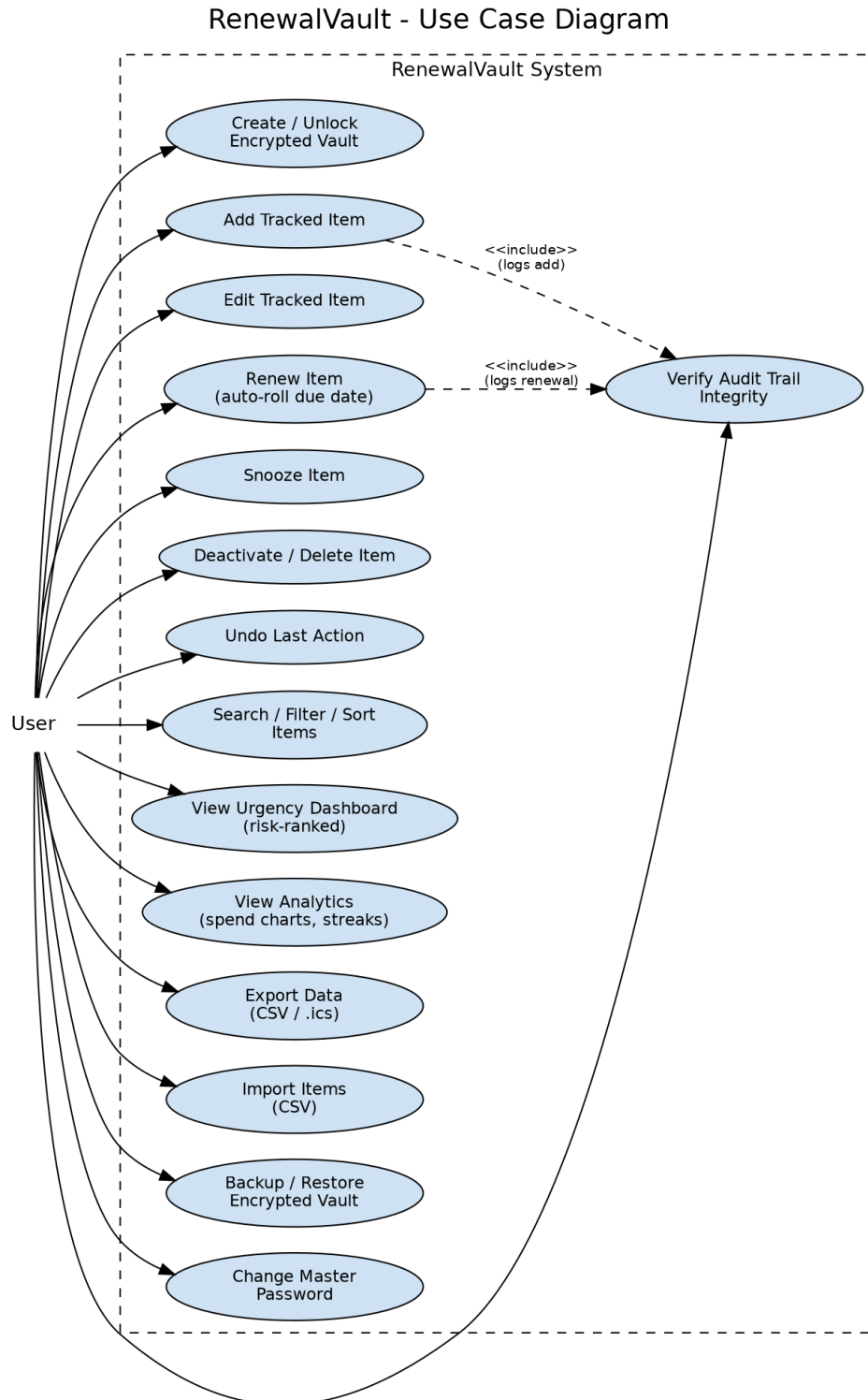


Figure 3: All 15 use cases available to the single actor (User), including the `<<include>>` relationship to audit logging.

6.3 Class Diagram

RenewalVault - Class Diagram (key classes)

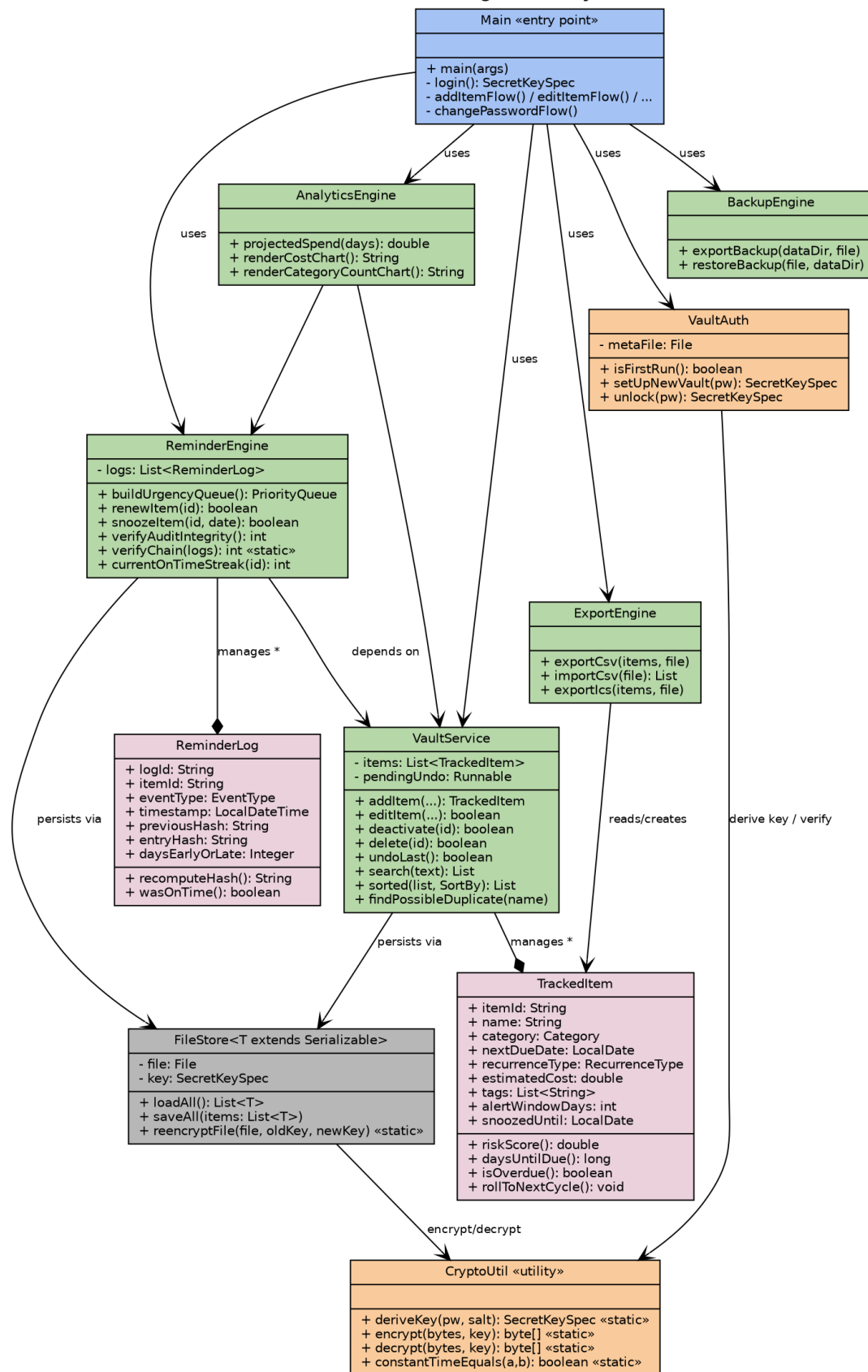


Figure 4: Key classes across all five packages, with their principal attributes, methods, and relationships.

6.4 Data Model / Schema Diagram

RenewalVault does not use a relational database; the brief's ER/schema requirement is addressed here as an ER-style view of the two encrypted, file-based record stores and how the vault's key material relates to them.

RenewalVault - Data Model / Schema Diagram
(file-based persistence, no relational DB -- ER-style view of the two encrypted record stores)

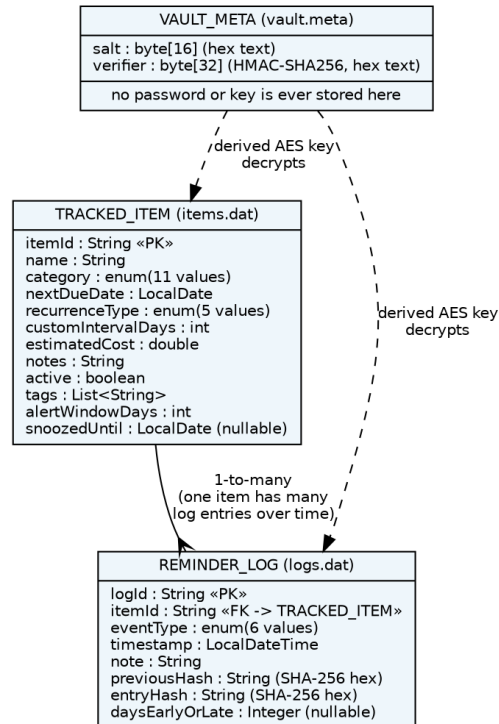


Figure 5: Data model for items.dat, logs.dat, and vault.meta, including the logical 1-to-many relationship between an item and its log history.

6.5 Sequence Diagrams

Two representative flows are shown: adding a tracked item (the core write path, touching encryption and the hash-chained log) and unlocking the vault (the core authentication path).

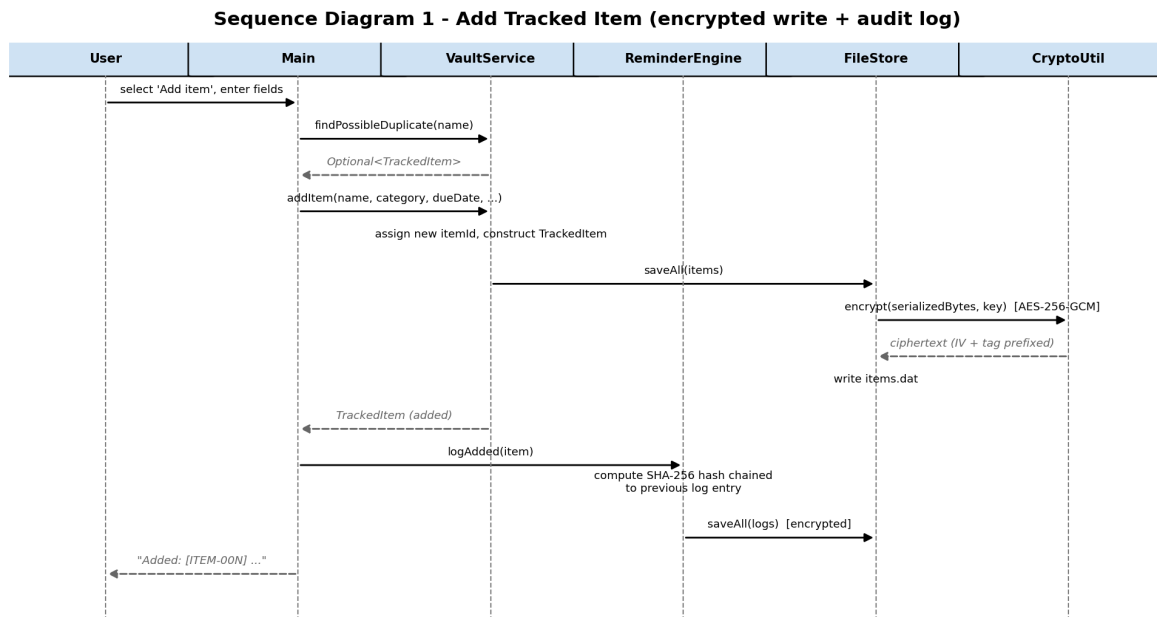


Figure 6: Sequence diagram for adding a tracked item.

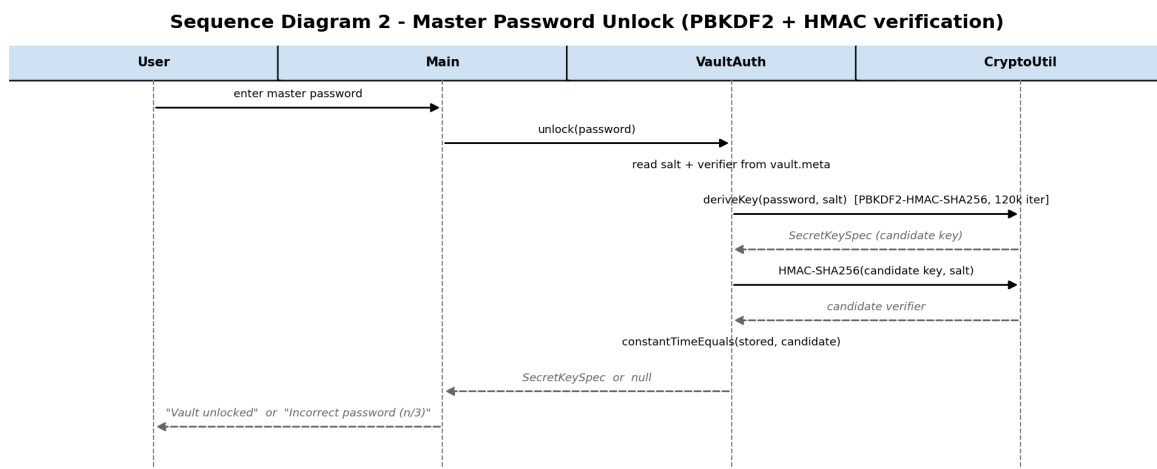


Figure 7: Sequence diagram for master password unlock (PBKDF2 + HMAC verification).

7. Design Decisions & Rationale

Why AES-256-GCM instead of AES-CBC

GCM is an authenticated encryption mode: it produces a built-in integrity tag, so any byte-level tampering with an encrypted file is detected automatically at decrypt time, with no separate MAC to implement or forget. This is what makes the tamper-detection demonstrated in Section 10 possible with almost no extra code.

Why PBKDF2 instead of using the password directly

A raw password is not a valid AES key, and using it directly (or a naive hash of it) would be vulnerable to brute-force and rainbow-table attacks. PBKDF2-HMAC-SHA256 with 120,000 iterations and a random 16-byte salt (per NIST SP 800-132 guidance) makes each guess computationally expensive and defeats precomputed tables.

Why a separate HMAC verifier instead of re-using AES-GCM for password checking

An earlier implementation attempted to verify the password by encrypting a known marker and hashing the ciphertext -- but AES-GCM's random per-call IV means the same key produces different ciphertext every time, so this check failed unpredictably even with the correct password. Switching to a deterministic HMAC-SHA256 verifier (same key and salt always produce the same output) fixed this. See Section 12 for how this bug was caught.

Why hash-chain the reminder log instead of just encrypting it

Encryption alone hides the log's content but does not prove its order or completeness -- an attacker with the key could still edit, delete, or reorder entries and re-encrypt. Chaining each entry's hash to the one before it (the same idea behind Git commits and blockchains) means any such edit breaks the chain at a provable, exact position.

Why a risk score instead of sorting by raw due date

Raw date order treats every item as equally important. A lapsed insurance policy or missed tax filing is categorically more consequential than a subscription due a day earlier. The risk score (urgency × category weight + cost factor) reflects that reality, and the category weights were chosen deliberately (COMPLIANCE_TAX and INSURANCE highest, GROCERY lowest) to match real-world stakes.

Why a CLI instead of a GUI

The brief allows any technically appropriate solution; a terminal interface keeps the project fully dependency-free (no GUI toolkit), trivially portable across operating systems, and keeps the focus on the underlying Java data structures, algorithms, and engine design that this course evaluates, rather than UI plumbing.

Why a hand-rolled test framework instead of JUnit

Adding JUnit would require a build tool (Maven/Gradle) or manually vendored jars, breaking the project's zero-dependency, 'just javac and java' constraint. A ~150-line assertion harness (TestRunner) provides the same pass/fail reporting with no external dependency.

8. Implementation Details

8.1 Core Java Concepts Demonstrated

Concept	Where it appears in RenewalVault
OOP: Encapsulation	Every model and engine class exposes state only through getters/setters or intention-revealing methods (e.g. <code>TrackedItem.riskScore()</code> , never raw field access)
OOP: Interfaces & enums with behaviour	<code>Category.riskWeight()</code> and <code>RecurrenceType</code> attach logic directly to enum constants rather than external if/switch chains scattered across the codebase
Generics	<code>FileStore<T></code> extends <code>Serializable</code> is a single, type-safe persistence class reused for both <code>TrackedItem</code> and <code>ReminderLog</code> with no code duplication
Collections Framework	<code>PriorityQueue</code> for the urgency queue, <code>List</code> for item/log storage, <code>Map</code> for category groupings in analytics
Streams API	Search, filter, sort (<code>VaultService</code>), category grouping and cost summation (<code>ReminderEngine.costByCategory()</code> , <code>Collectors.groupingBy/summingDouble</code>)
java.time API	<code>LocalDate/LocalDateTime</code> arithmetic drives all recurrence roll-forward logic and risk-score urgency calculations
Exception handling	Checked exceptions (<code>IOException</code> , <code>GeneralSecurityException</code>) are caught at well-defined boundaries (<code>FileStore</code> , <code>VaultAuth</code>) and surfaced as actionable CLI messages, never left to crash the session
File I/O & Serialization	Object serialization (<code>ObjectOutputStream/ObjectInputStream</code>) combined with <code>javax.crypto</code> for encrypted persistence; <code>java.util.zip</code> for the backup/restore feature
Functional interfaces / lambdas	<code>VaultService</code> 's undo mechanism stores a <code>Runnable</code> closure capturing prior state; <code>Comparator.comparingDouble(...).reversed()</code> drives every sort mode

8.2 Package Structure

```
src/main/java/renewalvault/  
|-- model/      TrackedItem.java, ReminderLog.java  
|-- engine/     VaultService, ReminderEngine, AnalyticsEngine,  
|              ExportEngine, BackupEngine  
|-- security/   CryptoUtil, VaultAuth  
|-- db/         FileStore<T>  
|-- cli/        Main, ConsoleColors  
`-- test/       TestRunner (12-case regression suite)
```

8.3 Key Algorithm: Risk Score

Each item's urgency is computed as:

```
urgency = overdue  
    ? 1000 + (daysOverdue * 25)           // overdue items dominate, and worsen over time  
    : max(0, 300 - (daysUntilDue * 6)); // decays as the due date recedes
```

```
costFactor = min(estimatedCost / 20, 100); // diminishing returns, capped
```

```
riskScore = (urgency * category.riskWeight()) + costFactor;
```

Category weights range from 2.0 (`COMPLIANCE_TAX`) down to 0.9 (`GROCERY`), so the same number of days-until-due produces a materially different, and realistic, priority.

8.4 Key Algorithm: Hash-Chained Audit Log

Every `ReminderLog` entry stores a SHA-256 hash of its own content concatenated with the previous entry's hash:

```
entryHash = SHA256(previousHash + logId + itemId + eventType  
                  + timestamp + note + daysEarlyOrLate)
```

`verifyChain()` walks the list once, recomputing each hash and confirming it both matches its own stored value *and* that the next entry correctly references it -- an $O(n)$ integrity check that returns the exact 1-based position of the first broken link, or -1 if the entire history is intact.

8.5 Key Algorithm: Recurrence Roll-Forward

On renewal, `TrackedItem.rollToNextCycle()` advances the due date using `java.time` arithmetic appropriate to the recurrence type (`plusWeeks/plusMonths/plusYears/plusDays`), and a `ONE_TIME` item instead deactivates itself, since it has no further cycle. Renewal punctuality is captured by comparing `daysUntilDue()` at the moment of renewal against zero, which feeds directly into the on-time-streak calculation described in Section 4's Reliability row.

8.6 Undo Mechanism

Rather than a full undo stack, `VaultService` keeps a single `Runnable pendingUndo` capturing a closure over the exact prior state of whatever was just changed (e.g. re-inserting a deleted item at its original index, or restoring every field an edit overwrote). This keeps the implementation simple while covering the realistic "I made one mistake, undo it" case the brief's usability expectations call for.

9. Screenshots / Results

The screenshots below are terminal-style renderings of real output captured by running the compiled application (the exact text and ANSI color codes shown were produced by actual runs of `renewalvault.cli.Main`; the login screen is shown as it appears with a real interactive terminal, which masks password input).

```
renewalvault - first run: vault setup

=====
RenewalVault - Never Miss a Deadline
(encrypted local vault + forensic audit trail)
=====

No vault found here.
1. Create a new vault
2. Restore from a .rvbackup file
Choose: 1

Let's set one up.
Choose a master password. It encrypts every file in ./data with AES-256 --
there is no recovery if you forget it, so pick something you'll remember.
New master password: *****
Confirm master password: *****
Vault created and encrypted.

=== UPCOMING DEADLINES (next 14 days + overdue, ranked by risk) ===
Nothing due soon. You're all caught up.
```

Result 1: First-run vault setup -- a new AES-256 vault is created from a master password.

```
renewalvault - item list (color-coded by urgency)

Choose an option: 3

=== ALL TRACKED ITEMS ===
[ITEM-001] Netflix Premium      | SUBSCRIPTION | Due: 2026-09-20 (+8 days) | DUE SOON | recurring | risk=284 #entertainment
[ITEM-002] Car Insurance Policy | INSURANCE    | Due: 2026-09-14 (+2 days) | DUE SOON | recurring | risk=618 #vehicle #essential
[ITEM-003] Annual IT Return Filing | COMPLIANCE_TAX | Due: 2026-09-30 (+18 days) | OK       | recurring | risk=384 #tax

--- MENU ---
1. Add a tracked item
2. Edit an existing item
3. List all items
4. Show urgency dashboard
5. Mark item as renewed
6. Snooze an item's alerts
7. Deactivate an item
8. Delete an item permanently
9. Undo last action
10. Search / filter / sort
Choose an option:
```

Result 2: Items listed with colour-coded urgency (yellow = due soon, green = OK).

```
renewalvault - analytics & budget projections

Choose an option: 13

=== ESTIMATED SPEND BY CATEGORY ===
INSURANCE      ##### 18500.00
SUBSCRIPTION   #      649.00
COMPLIANCE_TAX #       0.00

=== PROJECTED SPEND ===
Next 30 days : 19149.00
Next 90 days : 20447.00
Next 365 days: 26288.00

=== ON-TIME RENEWAL STREAKS ===
[HOT] Car Insurance Policy      3 renewal(s) in a row on time
```

Result 3: Analytics view -- category spend chart, cost projections, and on-time renewal streaks.

```
renewalvault - tamper detection in action

--- Scenario A: clean vault ---
Choose an option: 16
Audit trail intact: all 3 log entries are correctly hash-chained.

--- Scenario B: a single byte was corrupted directly on disk (logs.dat) ---
Master password: *****
Error: could not decrypt logs.dat. The master password may be wrong,
or the file has been tampered with.
Audit trail intact: all 0 log entries are correctly hash-chained.
(AES-GCM authentication rejected the corrupted ciphertext outright --
the app refuses to silently load tampered data.)
```

Result 4: Tamper detection -- a single corrupted byte in `logs.dat` is rejected by AES-GCM authentication instead of being silently loaded.

10. Testing Approach

Testing was done at two levels: an automated, zero-dependency regression suite (`renewalvault.test.TestRunner`), and manual end-to-end verification of the actual CLI.

10.1 Automated Regression Suite (12 cases)

No JUnit, Maven, or Gradle is used -- consistent with the project's zero-dependency constraint -- just a ~150-line hand-rolled assertion harness that runs entirely against disposable temporary directories (never the user's real vault). Actual output from a run performed for this report:

Running `RenewalVault self-tests...`

```
[PASS] AES-GCM encrypt/decrypt round trip
[PASS] VaultAuth: correct password unlocks, wrong password rejected
[PASS] Data at rest contains no plaintext item names
[PASS] Risk score ranks overdue + high-stakes items above low-stakes ones
[PASS] Recurrence roll-forward (weekly/monthly/yearly/custom/one-time)
[PASS] Hash chain: valid chain passes, broken link is detected
[PASS] CSV export/import round trip preserves item data
[PASS] Undo reverses add and delete
[PASS] Full encrypted backup/restore preserves the vault
[PASS] Duplicate item name is detected on add
[PASS] On-time renewal streak increments, late renewal resets it
[PASS] Master password rotation re-encrypts existing data correctly
```

12 passed, 0 failed.

Each case targets a specific correctness property:

- AES-GCM encrypt/decrypt correctness and that ciphertext never contains the plaintext
- Correct-password unlock and wrong-password rejection (VaultAuth)
- That encrypted files on disk contain no readable item names or notes
- Risk-score ordering: overdue and high-stakes categories outrank low-stakes ones due sooner
- Recurrence roll-forward arithmetic for all five recurrence types
- Hash-chain integrity: a valid chain verifies clean; a spliced-in tampered entry is caught at the exact position it occurs
- CSV export/import round-tripping, including names with embedded commas and quotes
- Undo correctly reversing both an add and a delete
- Full encrypted backup/restore preserving both the vault's password and its data
- Case-insensitive duplicate item name detection
- On-time renewal streak tracking (increments on time, resets on a late renewal)
- Master password rotation: the old password stops working and the new one correctly decrypts the re-encrypted data

10.2 Manual End-to-End Testing

- Adding, editing, and renewing items across multiple categories, recurrence types, tags, and custom alert windows via the real compiled CLI

- Restarting the application and confirming all items and full reminder history persist correctly across runs while encrypted at rest
- Changing the master password mid-session and confirming the old password is rejected on the next run while the new one unlocks the vault with all data intact
- Exporting a full encrypted backup and restoring it into a brand-new, empty directory, confirming the same password and data come back unchanged
- Deliberately corrupting a byte in an encrypted log file and confirming the tampering is detected instead of silently loaded (Result 4, Section 9)
- Exporting to CSV and re-importing it, and exporting to .ics and confirming the file is well-formed iCalendar (VCALENDAR/VEVENT structure, DTSTART, RRULE)

11. Challenges Faced

Non-deterministic password verifier

The first implementation of the master-password check encrypted a known marker with AES-GCM and hashed the ciphertext as the stored verifier. Because AES-GCM uses a random IV on every call, the same correct password produced a different verifier every time it was checked -- so even the right password intermittently failed to unlock the vault. This was caught during manual reload testing and fixed by switching to a deterministic HMAC-SHA256 verifier, which always produces the same output for the same key and salt.

Inverted early/late sign in streak tracking

The on-time renewal streak feature initially computed 'days early or late' with an inverted sign, so renewing an item several days *before* its due date was incorrectly logged as renewing it *late*. This was caught by the automated test suite (testRenewalStreak), not by manual testing -- a concrete example of why the regression suite earns its place in the project rather than being a formality.

CSV escaping for names containing commas and quotes

Item names can legitimately contain commas or quotation marks (e.g. a note like 'Renew before, or the policy lapses'). A naive CSV writer would corrupt such rows on re-import. A small quote-aware CSV parser/writer (RFC 4180-style escaping) was written by hand rather than pulling in an external CSV library, and is covered by the testCsvRoundTrip case.

Coordinating in-memory session state across a master-password rotation

Rotating the master password requires flushing current data under the *old* key, generating a new salt/verifier, re-encrypting existing files at the byte level, and then reloading every engine against the new key to confirm the rotation actually worked end to end -- rather than trusting that it worked. Structuring this as an atomic sequence (persist → rotate → re-encrypt → reload-and-verify) avoided leaving the vault in a half-migrated state if any step failed.

12. Learnings & Key Takeaways

- **Applied cryptography is unforgiving of subtle mistakes:** the AES-GCM verifier bug showed that a cryptographically sound primitive (AES-GCM) can still produce an incorrect system if used for a purpose (deterministic verification) that its design doesn't guarantee.
- **Automated tests catch bugs that manual testing misses:** two real, non-trivial bugs (the verifier issue and the streak sign inversion) were caught specifically by writing tests, not by using the application manually -- a direct, first-hand argument for regression testing rather than a textbook one.
- **Hash chaining is a small amount of code for a large integrity guarantee:** roughly 20 lines of hashing logic turn a plain log file into one that can prove, mathematically, whether it has been altered -- the same principle underlying chain-of-custody in digital forensics and the integrity guarantees of blockchains and Git.
- **Good layering pays off during feature growth:** because FileStore and CryptoUtil have no knowledge of what they're encrypting, adding BackupEngine and the entire on-time-streak feature in a second development pass required no changes to the encryption or persistence code at all.
- **A CLI's constraints are also design tools:** without a GUI framework to lean on, usability had to come from careful menu structure, colour-coding, and consistent prompts -- forcing clearer thinking about workflow than a drag-and-drop UI builder might have.

13. Future Enhancements

- A graphical (desktop or web) front-end reusing the existing engine layer unchanged, since the CLI layer is already isolated behind plain Java method calls
- Optional local push notifications (OS-level) as an alternative to relying on .ics import or manually opening the app
- Multi-user / multi-vault support, so a household could share selected categories while keeping others private
- A machine-learning-based cost forecaster trained on historical renewal data, extending the current linear projection in AnalyticsEngine
- A pluggable notification channel abstraction (email/SMS) for users willing to opt in to an external service, kept strictly outside the current offline-only core
- Biometric or hardware-key-backed unlock as an alternative to a memorized master password

14. References

- NIST Special Publication 800-132, *Recommendation for Password-Based Key Derivation* (basis for the PBKDF2 iteration-count and salt-length choices)
- NIST Special Publication 800-38D, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*
- Oracle Java Cryptography Architecture (JCA) Reference Guide -- `javax.crypto` API documentation
- RFC 5545, *Internet Calendaring and Scheduling Core Object Specification (iCalendar)* -- basis for the .ics export format

- OWASP Password Storage Cheat Sheet -- general guidance on salting, iteration counts, and avoiding plaintext password storage
- Oracle Java SE 21 Documentation -- `java.time`, `java.util.zip`, and `java.io` package references

End of Report -- RenewalVault, VITyarthi Build Your Own Project Submission